INFSCI 2160 Final Project
Kyle Meng
Kym64

<u>Significance statement:</u> The goal of this project is to see if there are certain words to avoid in order to write a politically center article and if there are easy ways to distinguish between the different media biases that appear in different articles.

## Introduction

This project uses the “News articles for political bias classification” dataset from Kaggle to explore the text/focus differences between left, right and center news sources. The project looks at 4 different aspects of the dataset.

1. Looking at words at appear frequently in 1 categorization vs the other
2. Looking to see if there are clean breaks in the text embeddings of different categorizations
3. Looking to see if there are already well performing existing models that can distinguish between the political bias of an article
4. Looking to see if there are ways to sufficiently distinguish between the categories.

## Data Introduction

The data contains 10832 news articles written by different new sources from 2014 to 2025. Each article is separated into 5 categories: left, leaning left, center, leaning right, and right. The categorization comes from a website called [allsides.com](allsides.com), which aims to present the same news story from multiple political perspectives and sources. For the sake of simplicity, I created a simple bias column where I grouped leaning left/left as left, and leaning right/right as right.

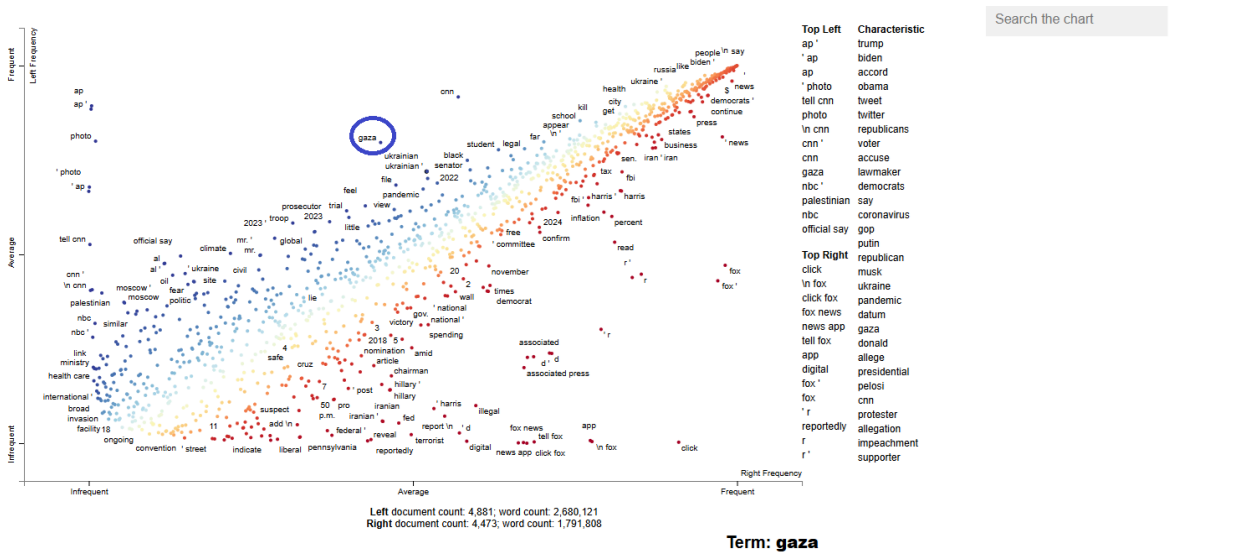
1-10 10,832 rows × 10 cols

| | url | topic | date | title | site | bias | page_text |
|---|---|---|---|---|---|---|---|
| | None Count 0 | None Count 0 | None Count 0 | None Count 0 | None Count 0 | None Count 0 | None Count 0 |
| | 10832 Unique values | politics 14.7%<br>elections 8.4%<br>Other 76.9% | 2025-04-03 0.2%<br>2025-05-22 0.2%<br>Other 99.7% | 10829 Unique values | Fox News Digital 15.8%<br>Washington Examiner 8.7%<br>Other 75.6% | right 27.6%<br>leaning-left 27.2%<br>Other 45.2% | 10832 Unique values |
| 0 | https://www.npr.org/202… | general-news | 2021-01-01 | What Got Us Through 202… | NPR (Online News) | leaning-left | What Got Us Through 202… |
| 1 | https://www.cbsnews.com… | general-news | 2020-12-31 | New Year's Eve gatherin… | CBS News (Online) | leaning-left | New Year's Eve gatherin… |
| 2 | https://nypost.com/2020… | general-news | 2020-12-31 | Here's how countries ar… | New York Post (News) | leaning-right | Here's how countries ar… |
| 3 | https://www.theguardian… | coronavirus | 2020-12-31 | Wisconsin health worker… | The Guardian | left | Police in Wisconsin sai… |
| 4 | https://www.foxnews.com… | coronavirus | 2020-12-31 | Wisconsin hospital empl… | Fox News Digital | right | A Wisconsin-based hospi… |
| 5 | https://www.newsmax.com… | defense-and-security | 2020-12-30 | Trump to Declassify Int… | Newsmax (News) | right | President Donald Trump … |
| 6 | https://www.npr.org/202… | politics | 2020-12-30 | Census To Miss Year-End… | NPR (Online News) | leaning-left | Census Missed Year-End … |
| 7 | https://nypost.com/2020… | politics | 2020-12-30 | Census Bureau set to bl… | New York Post (News) | leaning-right | The US Census Bureau is… |
| 8 | https://www.foxnews.com… | politics | 2020-12-30 | Hawley says he'll objec… | Fox News Digital | right | Sen. Josh Hawley, R-Mo.… |
| 9 | https://apnews.com/arti… | politics | 2020-12-30 | Trump's $2,000 checks s… | Associated Press | left | Trump push on $2K check… |

## Simple Word Frequency Methods

The code used to tokenize/clean the text and generate the visualization can be found in ScatterText.ipynb. For this task I first did some simple cleaning of the text where I removed stop

words, lemmatizations, and some punctuations using Spacy's language models. Then I used the ScatterText package to create a corpus of the cleaned text. ScatterText provides a cool visualization looking at the word frequency/precision of each word associated with any side. The more often that a word appears on one side but infrequently on the other, the stronger the association that word has with the one side. The strongest word associations would be the words that appear on the top left/bottom right of the graph. It's surprising but there aren't actually a lot of words in those areas of the graph, which seems to suggest that the topic/word choice is actually consistent across the political spectrum, but it is how they use it in combination with the topic that differentiates them. Some of the strongest association words are FOX, AP, CNN etc, which indicates that for a future pass, I should be removing all news company names from the article text. Both visualization html can be found on the github, which are fairly similar to each other, just that one is dense rank and one is not.



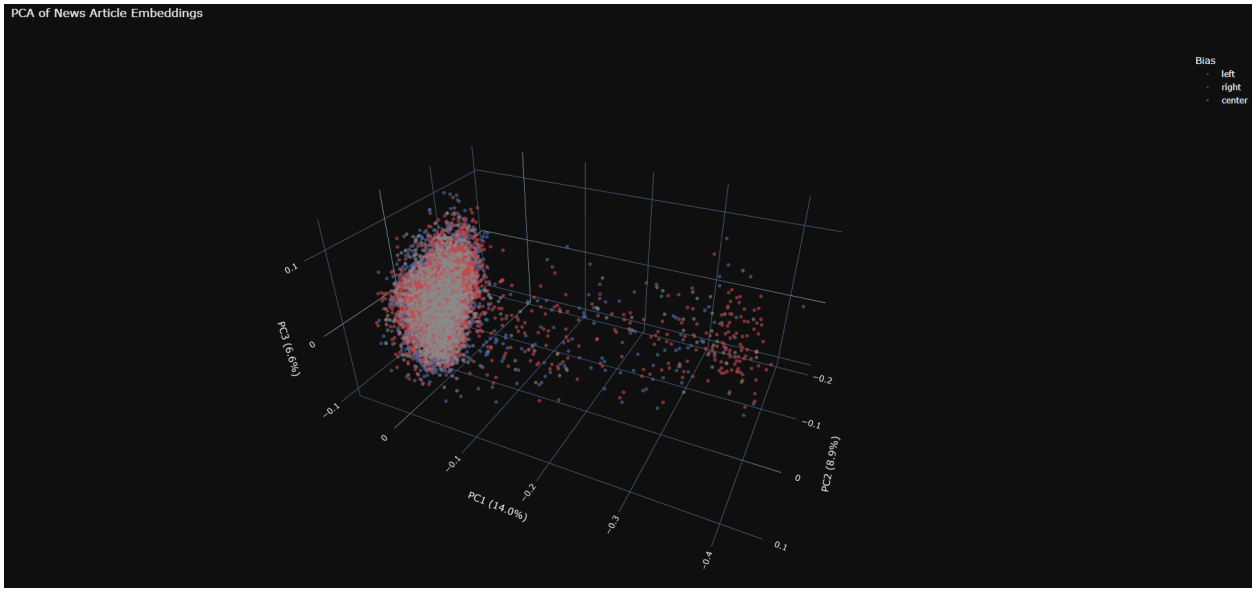

**Term: gaza**

**Left frequency:**
**20 per 25,000 terms**
**59 per 1,000 docs**
**The 2098 mentions in 288 available documents:**

**Right frequency:**
**10 per 25,000 terms**
**39 per 1,000 docs**
**The 674 mentions in 174 available documents:**

## Embedding Visualization

Next, I decided to convert the article text into text embeddings. To do so, I used the Longformer-base-4096 model found on Huggingface. I decided to use that model specifically because it takes in 4096 tokens as inputs, which is significantly more than a smaller 512 token model like all-MiniLM-L6-v2. In my mind, an article is likely longer than 512 words, so I felt more comfortable with the larger model. In terms of embedding time, the larger model took about 5 minutes to embed on GPU, vs ~25 seconds with MiniLM, which seemed reasonable. I then took the embedding and fitted it into a 3d PCA, and it accounted for ~30% of the variance. It's a good chunk of variance considering I started with a 700~ dimension embedding, but unfortunately 30% is still not as much as I would like. I found that there are no clean breaks in the 3d space between each of the biases, which somewhat lines up with what I found above.




Pretrained model evaluation and simple model training

Lastly, I wanted to see if an existing model on HuggingFace that can do these types of classifications already exists. I looked at the PoliticalBiasBert model, which is one of the most popular political bias classification models on the site. I did no finetuning, and had the model do zero shot classification on my data set. After scoring, I found the performance to be pretty bad.

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| center | 0.26 | 0.56 | 0.35 | 1478 |
| left | 0.57 | 0.33 | 0.42 | 4881 |
| right | 0.58 | 0.63 | 0.61 | 4473 |
| accuracy | | | 0.48 | 10832 |
| macro avg | 0.47 | 0.51 | 0.46 | 10832 |
| weighted avg | 0.53 | 0.48 | 0.49 | 10832 |

For the sake of tuning time, I didn't finetune the Huggingface model, but rather I decided to try and train 2 simpler models (logistic regression and XGBoost trees) to see if I can get a good classification model.

A naive logistic regression with default parameters had similar performance than the pretrained Huggingface model

```
Baseline Logistic Regression:
              precision    recall  f1-score   support

      center       0.73      0.09      0.16       296
        left       0.63      0.84      0.72       976
       right       0.73      0.67      0.69       895

    accuracy                           0.67      2167
   macro avg       0.69      0.53      0.52      2167
weighted avg       0.68      0.67      0.63      2167
```

I then tuned the XGBoost/Logistic regression model using hyperopt and found that the performance improved quite a bit.

```
Tuned XGBoost:
              precision    recall  f1-score   support

      center       0.79      0.48      0.60       296
        left       0.74      0.86      0.79       976
       right       0.81      0.77      0.79       895

    accuracy                           0.77      2167
   macro avg       0.78      0.70      0.73      2167
weighted avg       0.77      0.77      0.76      2167


Tuned Logistic Regression:
              precision    recall  f1-score   support

      center       0.76      0.50      0.61       296
        left       0.73      0.83      0.78       976
       right       0.80      0.77      0.79       895

    accuracy                           0.76      2167
   macro avg       0.77      0.70      0.72      2167
weighted avg       0.77      0.76      0.76      2167
```

## Conclusion

Overall I found the project to be interesting. Unfortunately from the results, it appears that there is no easy way for a human to determine the political bias of an article from just the word count alone, and even the topics that are brought up are mixed. Logically it makes sense, since at the end of the day, the articles are talking about the same events, just spun differently. This divide seems distinguishable for the more advanced models, but it's quite hard to get explainability out of using embeddings as features. In the following section, I've listed out a couple of things that I would've done differently/improved if I had more time for a future iteration.

## Potential Improvements for future

- Since the data set ranges from 2014-2025, it could be cool to divide it up into 2 different time periods (2014-2019 vs 2020-2025) and see how the frequent word usage has changed between the 2 time periods
- Better word filtering in the future, there are some common words that can be removed, or remove words like fox/cnn which is obviously indicative of what the media bias would be
- I can finetune my own classification model using huggingface models instead of just zero-shot predicting using a pretrained model. The xgboost/logistic regression models I trained are fairly simplistic and provide little explainability. There exists explainability methods (word importance) for HuggingFace models, but it just takes a long time to generate.

## Links:

- Kaggle Dataset: https://www.kaggle.com/datasets/gandpablo/news-articles-for-political-bias-classification
- Huggingface embedding model: https://huggingface.co/allenai/longformer-base-4096
- Pretrained Huggingface model for political bias classification: https://huggingface.co/bucketresearch/politicalBiasBERT
- Github code base: https://github.com/KyleMGit/INFSCI-2160-Final-Project